mOx 系統三階段開發與產品發展藍圖

本藍圖旨在說明 mOx 租賃系統之發展階段與架構預留要求。請貴團隊針對「第一階段 (Phase 1)」進行詳細的工時評估與報價，並確保資料庫 (DB Schema) 與底層架構能無縫銜接第二、第三階段之擴充。

🚩 第一階段：客戶 POC (本次報價與開發範圍)

🎯 專案目標：驗證核心交易閉環，實現「專人服務」與「自助取還 (IoT)」雙軌營運的最簡可行產品 (MVP)。

🛠️ 系統邊界：僅需完成 Happy Path 與基礎防呆，進階異常（如跨車種換車）允許透過客服手動退補款/重下單解決。極端事故則以「強制還車」截斷計費，後續營業損失轉由線下營運機制處理。

Phase 1 必須開發之核心功能 (MVP Scope)：

- 【系統共用模組】
  - 基礎後台權限 **(RBAC)**：至少需區分唯讀(View)、操作(Edit)、管理財務(Admin) 三種基礎帳號層級，確保財務與退款操作安全。
  - 基礎通知串接：
    - SMS 串接：僅用於「註冊與登入 OTP 驗證碼」，確保帳號實名安全。
    - Email 串接：自動發送「數位合約 PDF」與「藍新電子發票」副本給消費者，完成法律與財稅存證。
    - LINE OA API 串接：
      - 消費者端：發送即時訂單狀態（預約成功、取車提醒、還車結算通知）。
      - 營運商端 (Admin Alerts)：實作「營運 LINE 群組告警」，系統將新訂單、低電量告警、客服強還等關鍵事件拋送至內部群組。
- 【**OMS** 訂單模組】
  - 雙向建單系統：
    - **C**端 **(Web/PWA)**：提供消費者自主下單介面與會員訂單紀錄頁面。
    - **B**端 **(**後台**)**：
      - 提供門市內部 Key 單介面，支援極簡必填欄位 (僅手機與稱呼)、指定車牌下拉選單 (需自動過濾已預約車輛)。
      - 過渡性批次匯入 (Transitional Batch Import)：定位為 API 串接完成前之輕量級過渡工具。支援極簡 Excel/CSV 模板上傳，僅批次建立格式正確且有庫存之訂單；遇異常即略過並產出錯誤清單，不開發複雜內聯編輯。
  - 取還模式、數量與預約限制 **(Crucial)**：
    - 專人模式 **(**門市**)**：C端預約視窗為 T+1天 (最快隔日) 至 60天內；B端後台無此限制 (支援 T+0 當日建單)。允許 1 單多車；支援「訪客預訂」。
    - 自助模式 **(IoT)**：預約視窗為 T+10分鐘 至 7天內。嚴格限制「1 人 1 車」，且同時段不可有重疊訂單。

    - 提早取車與租期平移 **(Temporal Shift)**：支援 T-15m 提前解鎖取車。若提前取車，系統需自動將「實際起租時間」提前，並等距平移「預定還車時間」，確保總時數不變。
    - 數位合約與簽署 **(Digital Contract & e-Signature)**：將簽署節點前置於「結帳付款前」。初次下單須透過「數位簽名畫布 (Canvas)」親筆簽署；針對既有顧客（二次下單）實作「一鍵確認同意」機制並自動帶入歷史簽名檔。簽署完成後訂單即正式成立，並於 C 端會員介面提供「隨時調閱/下載」合約（包含歷史合約）功能。
    - 全域付款倒數 **(防惡意佔車)**：數位合約簽署完成、訂單成立後，統一給予 **10** 分鐘 的付款倒數時間。消費者須於 10 分鐘內完成金流付款與發票開立，超時未付系統自動作廢訂單並釋放庫存。
    - 雙階段金流與電子發票 **(NewebPay** 藍新 **API)**：
        - 線上結帳 **(**嚴格風控**)**：所有線上交易僅於第一次綁卡時需通過 **3D Secure** 驗證（行動支付則憑設備生物辨識）。結帳時實作信用卡「租金扣款 + 預授權保證金圈存」，並同步串接藍新電子發票 **API** 自動開立發票給消費者。
        - 線下結帳：B端後台需支援選擇「門市現金/匯款/記帳」等免預授權與手動發票選項。
    - 基礎訂單狀態：實作 Unpaid → Paid → Active → Pending_Settlement → Completed / Cancelled，及內部 Void(作廢)/Allowance(折讓)。
    - 數位車況點交 **(UGC** 影像存證**)**：開發 C 端消費者於取還車流程中之「車況照片/影片上傳」功能，以及 B 端後台對應的影像調閱介面（檔案可直傳雲端或指定 NAS）。
    - 客服控制台 **(Remote Assist)**：提供後台介面發送遠端指令排解異常，並必須支援「強制還車」按鈕，以利客服在遇到極端車禍事故或斷訊時，能隨時截斷計費停損。
- 【**Asset** 資產模組】
    - 單一硬體 **API** 串接：透過 API 串接車載 IoT (包含讀取里程數、剩餘能源)。遠端開/關鎖機制，前端必須實作「密碼輸入驗證」以防盜用。
    - 雙軌庫存防撞 **(**硬鎖定**)**：確保預約單在時間軸上絕對不與即時單衝突，並落實 No-Show 絕對保留庫存。
    - 基礎站點追蹤：支援 A-to-A 與 A-to-B 模式，訂單結束後更新車輛站點歸屬。
    - 手動內部調度與照片紀錄：支援 B 端後台人員手動更改車輛之歸屬站點，並開發「上傳移機確認照片」之功能作為內部紀錄。
- 【**Pricing** 計費模組】
    - 基礎計費引擎：時租/日租計算、24H 週期封頂邏輯、00:00 假日費率嚴格切割。
    - 逾時與規費結算：落實「15 分鐘寬限期」邏輯（向下取整）。進入 Pending_Settlement 時，系統自動帶入里程費，且 B端後台需提供表單介面供人員手動輸入 ETC、清潔費、車損等附加費用後，方發動預授權請款。
    - 附加服務：加購項目（如兒童座椅：$200/日）。計價週期隨同主訂單之 24H 週期
    - 行銷折扣：折扣碼折抵，計算後最低結算總金額為 $0（不產生負數）
    - 提早還車不退費政策：防範惡意佔車，提早歸還一律不退還未使用之租金與附加服務費。
- 【**CRM** 風控模組】
    - 基礎身分識別：OTP 手機號碼註冊與登入。
    - **KYC**：支援用戶上傳證件照片，由後台人員「手動點擊審核」放行。

🚫 Phase 1 明確排除範圍 (Out of Scope for Phase 1)：

- ❌ 預測性容量防爆檢核 (預估 = 現有+還入-取走)。

- ❌ 智慧派車引擎 (如：低電量優先派車演算法)。
- ❌ 多供應商自動備援切換與 MQ 佇列微服務 (Phase 1 串接單一發送商即可)。
- ❌ 自動化影像 N+1 滾動刪除機制 (影像生命週期初期由人員手動管理，不開發自動化刪除排程)。
- ❌ 內部調度移機之「自動化 GPS 圍籬防呆驗證」(僅需實作上述之手動改站與上傳照片即可)。
- ❌ 產險公司 Webhook API 對接。
- ❌ 自動化線上續租與預授權追加 (Phase 1 遇續租展期，由客服手動處理或另建新單)。
- ❌ 跨車種升降級、同級換車之自動化財務退補運算。
- ❌ 自動化長租套餐與特殊費率覆寫 (初期遇長租客，先由門市人員透過「手動折扣」處理)。
- ❌ 自動化安全性帳號遷移與活體辨識 (初期變更手機號碼由客服人工核身處理)。
- ❌ 系統內建之自動化事故營業損失計算 (遇事故直接以強制還車截斷，損失求償轉由線下處理)。

🚩 第二階段：規模化擴張 (架構預留指標)

(註：本階段暫不報價，但要求 *Phase 1* 的資料庫架構必須滿足以下擴充性)

- 多租戶架構 **(Multi-tenant)**：底層資料庫需預留 tenant_id，以支援未來 3-5 家獨立車行之資料隔離。
- 營運突發異動引擎：支援訂單執行中進行同級換車、續租展期之預授權追加。
- 信用評分與權限連動：資料庫需預留 credit_score 欄位，未來將實作低分自動剝奪自助取車權限。
- 長租套餐與特殊費率覆寫引擎：系統自動判定租期天數，套用最優惠的週租/月租公式。
- 離線二次結算：針對 IoT 斷訊引發的強制還車，支援「先扣已知費用、尋回車輛再扣差額」。
- 通知系統升級：實作 SMS/Email 的自動備援切換 (Failover) 與通知發送排程佇列。

🚩 第三階段：生態系與 API 串接 (架構預留指標)

- **Open API** 接口：系統架構需支援開設 B2B API，供 Klook/KKday 即時抓取庫存與拋轉訂單。
- 動態定價 **(Dynamic Pricing)**：支援外部參數 (如天氣、連假) 覆寫預設費率。
- 聯盟清算系統：支援跨店票券 (Coupon) 抵扣與平台補貼財務對帳。

💡 開發團隊注意事項 (Development Guidelines)

1. 資料庫一次到位：
    - 強烈建議在規劃 Phase 1 之 DB Schema/ER Model 時，將 Phase 2 & 3 提及之實體關係（如多租戶、站點容量、信用評分、折扣券關聯）一併考量，避免未來發生破壞性重建。
    - 雖然 Phase 1 是人工審核駕照，但 User 表中必須預留 `license_expiry_date` 欄位，方便 Phase 2 直接開啟自動巡檢功能。
2. 金流與硬體解耦：金流 Gateway 與 IoT API 串接層，請採用 Adapter/Interface 設計模式，以利未來抽換支付服務商或更換硬體設備商。
3. 混合通知：雖然只串接基礎 API，但系統必須具備將訊息分類並派發至不同管道的 NotificationHandler 邏輯。